

#1 Contains Duplicate

Arrays & Hashing

LeetCode 217

NeetCode "Duplicate Integer"

PROBLEM

Given an integer array `nums`, return `true` if any value appears at least twice in the array, and return `false` if every element is distinct.

```
nums = [1,2,3,1] → true
```

```
nums = [1,2,3,4] → false
```

```
nums = [1,1,1,3,3,4,3,2,4,2] → true
```

CORE IDEA

Goal

Determine whether any value repeats in the array.

Key insight

A hash set gives $O(1)$ average-case lookup and insertion, so a single pass can catch a repeat the moment it's seen, with no nested loops.

Trigger: "Have I seen this value before?" → reach for a Hash Set membership check.

OPTIMAL SOLUTION

Data structure — Hash Set (Python `set`)

1. Initialize an empty set `h`.
2. Iterate through each `num` in `nums`.
3. If `num` is already in `h`, return `True` immediately — duplicate found.
4. Otherwise, add `num` to `h` and continue.
5. If the loop finishes with no match, return `False`.

$O(n)$

TIME

$O(n)$

SPACE

CODE

```
class Solution:
    def hasDuplicate(self, nums: List[int]) -> bool:
        h = set()
        for num in nums:
            if num in h:
                return True
            else:
                h.add(num)

        return False
```

TIPS TO REMEMBER

Mental shortcut — "Seen it before? Set it and forget it."

Common / related — Contains Duplicate II (nearby-index variant), Contains Duplicate III (nearby-value variant), Longest Consecutive Sequence, Single Number

"I'm trading $O(n)$ space for $O(n)$ time with a hash set, which avoids the $O(n^2)$ brute-force approach of comparing every pair."

EDGE CASES

- ▶ Empty array → no duplicates possible, return `False`
- ▶ All elements identical → returns `True` as soon as the second element is checked
- ▶ Single-element array → return `False`
- ▶ Negative numbers and zero → ints are hashable as-is, no special handling needed